A Synopsis

Of

Project

On

DeskPilot

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Dr. Akansha Gupta
(Professor)

Project Team ID: MP2026CSE58

Submitted By

Name of Student1: Naman Kumar	**Enrollment No.: GE-232023472**
Name of Student2: Abhishek Sharma	**Enrollment No.: GE-232023744**
Name of Student3: Anshika Rawat	**Enrollment No.: GE-232023841**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GRAPHIC ERA DEEMED TO BE UNIVERSITY
DEHRADUN

SEPTEMBER 2026

CERTIFICATE

This is to certify that the project synopsis entitled **"DeskPilot"** is being submitted by **Naman Kumar(GE-232023472), Abhishek Sharma(GE-232023744) and Anshika Rawat(GE-232023841)** for the partial fulfillment of the requirement for the award of the degree of **B. Tech-CSE** in Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, and is a record of his/her own work carried out under my supervision. The matter embodied in the synopsis has not been submitted in any other university or institution for the award of any degree.

Name of the Supervisor : Dr. Akansha Gupta
Designation : Professor
Department of Computer Science and Engineering,
Graphic Era Deemed to be University, Dehradun, India

DECLARATION

I/we hereby declare that the project synopsis entitled "**DeskPilot"** is being submitted for the partial fulfillment of the requirement for the award of the degree of **B. Tech-CSE** in Department of Computer Science & Engineering, Graphic Era Deemed to be University, Dehradun, is my/our original work and has not been submitted to any other university or institution for the award of any degree or diploma.

Naman Kumar (23011020) **Signature** 

Abhishek Sharma (230213914) **Signature** 

Anshika Rawat (230223817) **Signature** 

TABLE OF CONTENTS

S.No.	CONTENT	PAGE NO.
1.	Abstract	5
2.	Introduction	6
3.	Literature Review	7-9
4.	Problem Statement & Objectives	10
5.	Proposed Methodology	11
6.	Workflow Architecture	12
7.	Hardware and Software Requirements	13
8.	Expected Outcomes	14
9.	Conclusion	15
10.	References	16

ABSTRACT

Everyday computer tasks such as merging documents, reformatting spreadsheets, or extracting information from files are repetitive but still require manual effort. Automating such tasks currently relies on large, cloud-hosted language models, raising concerns around data privacy, since files may contain sensitive information, and around resource use, since simple tasks are processed using disproportionately large models. This project proposes a lightweight, locally deployable system that uses a small language model to interpret user instructions in plain language and identify the required file operation. Actual execution, such as merging images into a document or formatting spreadsheet data, is handled by a fixed set of reliable local functions built for each supported file type, keeping the model's role limited to understanding intent. By processing everything on the user's own device, the system eliminates the need for an internet connection, protects user data, and reduces computational overhead, offering a practical, privacy-preserving alternative to online models for routine file-related tasks.

INTRODUCTION

Everyday computer use often involves repetitive file-related tasks, merging documents, converting formats, extracting information, or reorganizing spreadsheet data. While each task is simple to describe, completing it manually still takes time and requires familiarity with specific software. Recent advances in language models have made it possible to describe such tasks in plain language and have a system perform them automatically. However, most tools offering this capability rely on large, cloud-hosted models, which introduces two key concerns. First, privacy: many files contain personal or sensitive information, and sending them to an external server raises legitimate confidentiality concerns. Second, efficiency: large models consume significant computational resources for every request, regardless of task complexity, making them a disproportionate and often costly choice for simple operations like merging images or renaming files.

Since most everyday file tasks are repetitive and follow predictable patterns, it is worth examining whether they truly require large, general-purpose online models, or whether a smaller, locally run model paired with a defined set of supporting tools can achieve the same result. If a compact model is used only to interpret user intent, while dedicated local functions handle the actual file operations, the drawbacks of online dependency can be avoided without losing the convenience of natural language instructions.

This project explores that possibility by proposing a lightweight, locally deployable system that interprets everyday instructions and performs common operations across formats such as Word, Excel, PDF, and images, entirely offline, preserving privacy and reducing computational cost.

LITERATURE REVIEW

3.1 Existing Problem

Everyday file-handling tasks, merging documents, reformatting spreadsheets, extracting information from PDFs, and organizing images, are simple to describe but still demand manual effort or reliance on large, cloud-hosted language models. Brown et al. [2] showed that strong general-purpose language understanding, of the kind needed to follow plain-language instructions, was only achieved by scaling models to hundreds of billions of parameters, and Ouyang et al. [3] further showed that making such models reliably follow human instructions required an additional, resource-intensive fine-tuning stage (RLHF) on top of that scale. Together these works establish why current instruction-following assistants are large and cloud-hosted: the capability the project needs was historically inseparable from massive scale. This creates two problems for routine file tasks: user files, some containing personal or confidential data, must be uploaded to an external server to be processed, and a disproportionately large amount of computation is spent on operations, such as renaming a file or merging two images, that are logically simple.

3.2 How the Solution Was Identified

Investigating whether this dependence on scale was truly necessary led to two related bodies of work. First, Schick et al. [6] (Toolformer) demonstrated that a language model does not need to perform a task itself; it can instead learn, in a self-supervised manner, when to invoke an external tool or API and simply pass on the correct arguments, leaving execution to that tool. Patil et al. [7] (Gorilla) extended this idea, showing that a language model can be trained to reliably select and correctly format calls to a very large number of external APIs, using retrieval-augmented constraints to keep the output valid. Read together, these two works reframe the problem: the language model's job can be narrowed to intent recognition and parameter extraction, while a separate, deterministic layer performs the actual work. Second, Touvron et al. [4], [5] (LLaMA and Llama 2) and Frantar et al. [8] (GPTQ) showed that open, comparatively small foundation models, further compressed through post-training quantization, can run on ordinary consumer CPUs or GPUs while retaining usable language understanding. Combining these two findings suggested the specific direction this project takes: a small, quantized, locally run

model restricted to interpreting user intent, paired with a fixed set of deterministic local tools that carry out the file operations.

3.3 Existing Solutions

Three categories of existing work were studied in detail to identify what could be reused and what gap remained.

Robotic Process Automation (RPA). van der Aalst et al. [1] studied rule-based automation platforms such as UiPath and Microsoft Power Automate, which execute repetitive digital tasks reliably and deterministically. What is useful here is exactly this reliability and predictability of execution; what is missing is any natural-language understanding, since these platforms require the workflow to be predefined step by step and cannot interpret a plain-language instruction. This project reuses the principle of deterministic, rule-based execution but replaces the rigid, manually configured trigger with a language-model-driven intent parser.

Cloud-hosted LLM assistants. Brown et al. [2] and Ouyang et al. [3] were studied to understand how present-day assistants embedded in office suites achieve strong instruction comprehension: through large-scale pretraining followed by supervised and reinforcement-learning-based instruction tuning. This confirms that the comprehension quality users expect is achievable, but only at a computational and infrastructural cost that assumes constant server-side, internet-connected inference. This project takes the target level of instruction comprehension as its benchmark while deliberately restricting the model's role to keep it small enough for local inference.

Small and quantized local language models. Touvron et al. [4], [5] were studied for how open foundation models can be trained to be competitive at far smaller parameter counts than GPT-3, and Frantar et al. [8] were studied for how 4-bit post-training quantization compresses such models further with minimal loss in accuracy. The Ollama documentation [9] was studied as a practical runtime that packages quantized models for local CPU/GPU inference through a simple local API, removing the need to build a custom inference pipeline. This project directly adopts a quantized, instruction-tuned model of this kind, served through Ollama, as the intent-interpretation component.

Tool-augmented language models. Schick et al. [6] and Patil et al. [7] were studied for their methods of constraining a language model's output so that it reliably resolves to a valid

function call with correctly typed parameters, rather than free-form text. This project implements an analogous constrained-output/validation layer, adapted from the intent representation used here to the specific file-operation tools (Word, Excel, PDF, and image functions) this system supports.

Across these three areas, no existing work was found that combines a small, locally deployed language model restricted to intent parsing with a deterministic, RPA-style tool set built specifically for offline document, spreadsheet, image, and PDF automation. This is the gap the proposed system addresses.

3.4 Proposed Solution

Building on the above, this project proposes DeskPilot: a small, quantized, instruction-tuned language model (following [4], [5], [8], deployed via [9]) is used only to interpret a plain-language instruction and match it to a supported operation and its parameters, in the manner shown feasible by [6] and [7]. Execution itself is handed off to a fixed set of deterministic local functions, one per supported file operation, in the spirit of the reliable, rule-based execution studied in RPA platforms [1], but triggered by natural language instead of a predefined workflow. Because both the model and the tools run entirely on the user's device, the system avoids the internet dependency, privacy exposure, and computational overhead identified as the core problem in Section 3.1, while still offering the natural-language convenience associated with cloud-hosted assistants.

PROBLEM STATEMENT & OBJECTIVES

Everyday computer tasks such as merging files, reformatting spreadsheet data, extracting information from documents, or organizing images into a structured file are repetitive and simple in nature, yet still require manual effort or reliance on large, cloud-based language models to automate. This dependence on online models raises two key concerns: sensitive files must be transmitted to external servers, compromising user privacy, and substantial computational resources are consumed even for trivial tasks, making the approach inefficient and often costly. There is a need for a locally deployable solution that can understand natural language instructions and perform such tasks without depending on internet connectivity or large-scale models.

Objectives :

1. To design a system that uses a small, locally run language model to interpret user instructions written in plain language and identify the corresponding file operation.
2. To develop a set of reliable, deterministic tools for common file operations across formats such as Word, Excel, PDF, and images.
3. To integrate the language model with these tools so that user intent is accurately mapped to the correct operation and parameters.
4. To ensure all processing occurs entirely on the local device, preserving data privacy and eliminating dependency on an internet connection.
5. To evaluate the system's accuracy and efficiency in performing routine file-handling tasks compared to existing solutions.

PROPOSED METHODOLOGY

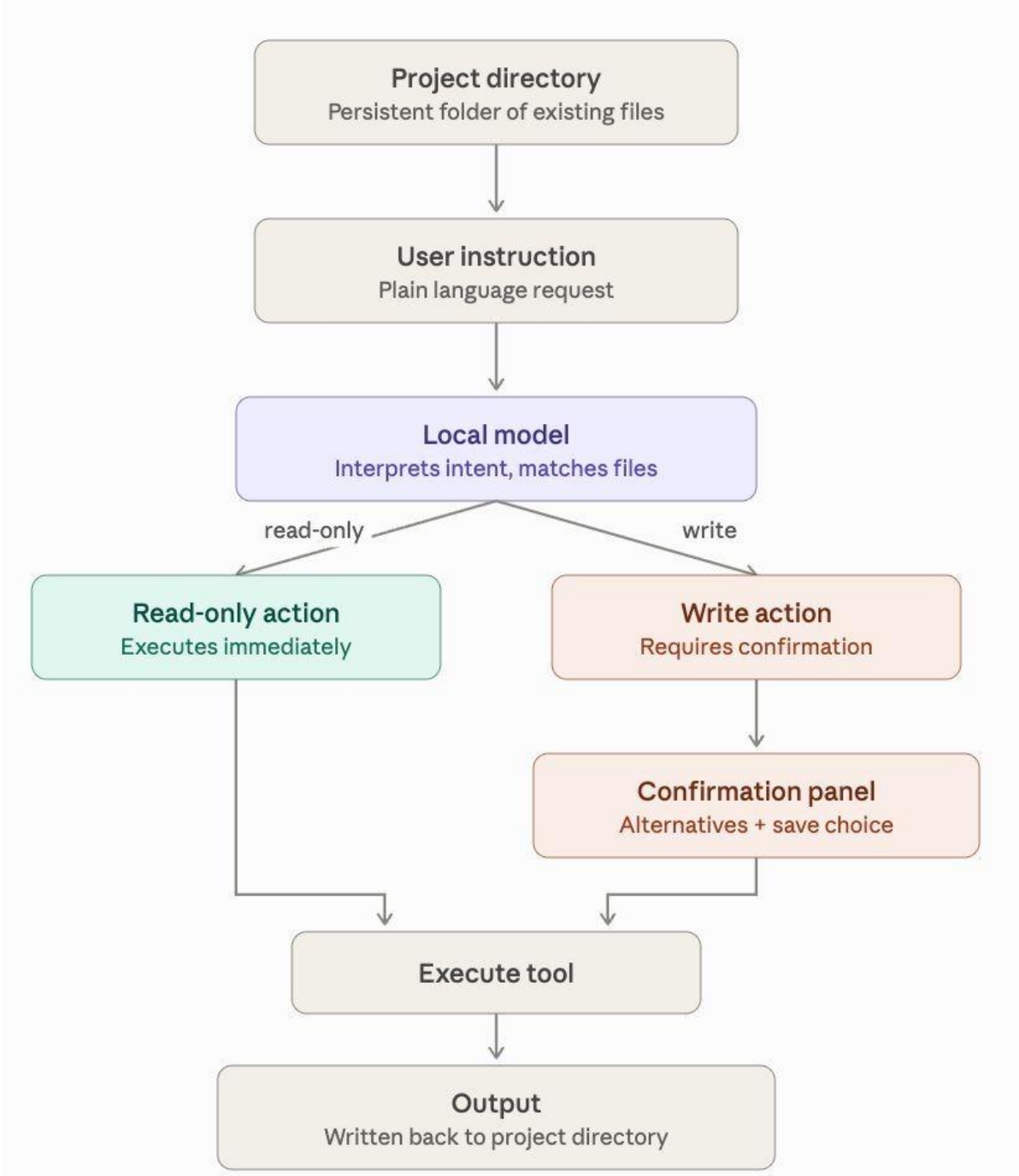
The system will be developed in three main stages. In the first stage, a small, instruction-tuned language model capable of running locally, such as a quantized variant compatible with CPU or limited GPU environments, will be selected and configured. This model will be responsible solely for interpreting user instructions and converting them into a structured representation of intent, rather than directly generating or modifying file content.

In the second stage, a set of deterministic tool functions will be developed for common file operations, including merging and formatting Word documents, editing spreadsheet data, extracting content from PDFs, and combining or organizing images. Each tool will accept clearly defined parameters derived from the model's output.

In the third stage, an integration layer will connect the language model to the tool set, using constrained output generation to ensure the model's responses reliably map to valid operations and parameters. This layer will validate the extracted intent, execute the corresponding function, and handle errors or ambiguous instructions.

The complete system will then be tested on a range of representative tasks to evaluate its accuracy, reliability, and processing efficiency in an offline environment.

WORKFLOW ARCHITECTURE



HARDWARE AND SOFTWARE REQUIREMENTS

Component	Basic Requirement
Computer	A laptop or desktop computer with a modern 64-bit processor.
Memory	At least 8 GB RAM for basic study; 16 GB or more is preferable for running local models.
Storage	At least 10 GB of free storage for software, model files, and project outputs.
GPU (optional)	A compatible GPU with sufficient VRAM improves local inference speed; smaller models may also run on a CPU.
Operating System	Windows, macOS, or Linux.
Software	Python 3.x, Jupyter Notebook or VS Code, and an LLM runtime such as Ollama; a hosted LLM interface or API may be used as an alternative.
Internet	Required for hosted services and initial model downloads; optional after a local model is downloaded.

EXPECTED OUTCOMES

- A working, locally deployable system that interprets natural language instructions and performs common file-handling tasks without requiring an internet connection
- Support for routine operations such as merging documents, formatting spreadsheet data, extracting information from files, and organizing images into structured outputs
- Demonstration that a small, locally run language model, when paired with a defined set of dedicated tools, can reliably perform well-defined tasks with accuracy comparable to solutions using large online models
- Improved data privacy, since all files and user instructions remain on the local device throughout processing
- Reduced computational and monetary cost compared to using large-scale cloud-hosted models for simple, repetitive tasks
- A modular tool framework that can be extended to support additional file formats or operations in the future
- A practical, resource-conscious alternative to existing automation tools, suitable for use in offline or resource-limited environments

CONCLUSION

Everyday computer tasks involving documents, spreadsheets, and images are often repetitive and simple in nature, yet automating them currently depends heavily on large, cloud-hosted language models. This dependence raises genuine concerns around data privacy, since sensitive files must be sent to external servers, and around efficiency, since substantial computational resources are used even for trivial operations. This project addresses these concerns by proposing a lightweight, locally deployable system in which a small language model is used only to interpret user instructions, while the actual file operations are carried out through a set of reliable, dedicated tools running entirely on the local device. By separating language understanding from task execution, the system aims to remain accurate and predictable while eliminating the need for internet connectivity. If successful, this project will demonstrate that small, locally run models can effectively automate routine digital tasks, offering a practical, privacy-preserving, and resource-efficient alternative to existing online-dependent automation solutions.

REFERENCES

[1] W. M. P. van der Aalst, M. Bichler, and A. Heinzl, "Robotic Process Automation," Business & Information Systems Engineering, vol. 60, no. 4, pp. 269–272, 2018.

[2] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 1877–1901.

[3] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems, vol. 35, 2022, pp. 27730–27744.

[4] H. Touvron et al., "LLaMA: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[5] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.

[6] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," arXiv preprint arXiv:2302.04761, 2023.

[7] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," arXiv preprint arXiv:2305.15334, 2023.

[8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[9] Ollama, "Ollama documentation," Ollama. [Online]. Available: https://ollama.com/